

CHEQ Churn Insights MCP

AI Engineer home assignment - one-page design

DESIGN

I built a local MCP server that lets Codex or Claude Code answer questions about the supplied Telco Customer Churn dataset. It loads the pinned 7,043-customer snapshot into DuckDB and supports counts, churn rates, averages, filters, and comparisons such as churn by contract. The server never accepts raw SQL; it runs only supported calculations and returns the data version and filters used.

FLOW

User question -> FastMCP exposes the tool contract (available tools and inputs) -> the agent can call `describe_dataset` for fields and limits -> it calls `analyze_customers` -> request schemas check the input, domain rules resolve approved values such as `unclear_reason`, and the analytics service runs DuckDB -> `server.py` returns the answer and source details.

EXAMPLE

"How many customers churned because they did not know why?" maps to churned customers with `unclear_reason`. That domain rule maps to the controlled label "Don't know"; DuckDB counts the rows and returns "130 customers."

WHY THIS SHAPE

This is a structured customer snapshot, so the useful questions are counts, segments, and rates - not document search. DuckDB is a columnar analytics database that runs inside Python. With 7K rows, it is fast and avoids PostgreSQL, a separate service, and Docker Compose. Python has strong data tools, and FastMCP handles MCP wiring and tool definitions. Go could also serve MCP, but would add code without a useful advantage here. I avoided RAG/vector search, a second agent framework such as LangChain or LangGraph, and LLM-written SQL. Codex or Claude Code already supplies the conversational LLM, while this dataset has controlled fields rather than free text.

PRODUCTION

I would package the server as a Docker image and expose it as a remote HTTPS MCP service. I would begin with Amazon ECS on Fargate for managed containers and autoscaling; I would choose EKS only if CHEQ already had a Kubernetes platform or needed its controls across many services. A small versioned snapshot could remain in S3; larger or live data would come from governed warehouse or CRM sources with freshness checks. Each request would carry an OIDC token, and the service would enforce role, tenant/customer scope, and purpose. Only approved roles could discover customer IDs or read a customer record.

RISKS AND CONTROLS

Unsupported questions receive a clear refusal, not a guess. Pinned data and schema checks catch source changes. The default server is aggregate-only: it returns no IDs and suppresses very small groups. The local trusted mode is not RBAC; production adds identity, audit logs, rate limits, encryption, and privacy review.

BUSINESS IMPACT

Customer Success, Marketing, and RevOps get one governed definition of churn, faster answers, and explicit customer-data access boundaries.